
Detecting Simplification in Neural Network Dynamics from Scalar Logs: Active Dimension in Controlled Recurrent Systems

Anonymous Author
Anonymous Institution

Abstract

Neural network training can restrict parameter motion, reduce latent information, or produce more regular behavior. We introduce a procedure for detecting such changes from one scalar time series using delay reconstruction, pooled local-dimension estimation, and fixed decision rules. Dynamical-systems theory motivates an active-component interpretation under suitable recurrence and observation conditions; synthetic experiments separate independent phases from harmonics. Applied tests track training restrictions in CNNs, information loss and preservation in a text VAE, and changes in learned behavior. Beyond diagnosis, the scalar estimate selects forecasting models for recurrent-network activity and policies for changed actuator dynamics. A 128-query approximation preserves all 160 full-estimator forecast choices on a separate 20-seed cohort. These results connect scalar geometry to useful decisions with low data requirements and measured computational savings. Absolute dimension recovery requires the stated assumptions; practical accuracy and cost depend on the observable, decision task, and competing baseline.

1 INTRODUCTION

A scalar training log records more than whether a loss or parameter norm rises or falls. Its fluctuations can reflect several interacting degrees of freedom. As learning progresses, some of these may disappear or become coordinated. Detecting that change from the log would

help monitor training without storing every parameter vector or repeatedly analyzing the full network state.

We develop a method for detecting such changes from one scalar time series. The starting point is delay-embedding theory: under suitable assumptions, a vector of delayed observations preserves the geometry of the underlying dynamics (Takens, 1981; Sauer et al., 1991; Stark, 1999; Stark et al., 2003). A dimension estimator applied to these vectors can then measure how many independent local coordinates the dynamics require. We use the pooled MacKay–Ghahramani (MG) form (MacKay and Ghahramani, 2005) of the Levina–Bickel estimator (Levina and Bickel, 2004).

In applied training, we test whether a decrease in MG accompanies restricted parameter motion, loss of latent information, or more regular learned behavior. We measure each change independently and compare MG between matched conditions with the same observable and estimator settings. This tests whether MG detects the specified simplification even when stochasticity or transients prevent a theoretically guaranteed dimension interpretation. Section 2.4 states the conditions for dimension estimation and for empirical monitoring.

Contributions.

1. **A method for monitoring simplification from scalar logs.** We combine MG with a specified observation and windowing procedure, numerical checks, and a change detector that uses only past data. Once the log is available, analysis depends on the window and embedding size, not the number of model parameters.
2. **Synthetic tests of the active-component interpretation.** We test recovery in analytic oscillators and learning problems, including an MLP-based task. In trained recurrent generators, MG from one neuron separates independent phases from harmonics even when the number of spectral peaks and covariance ranks are similar.
3. **Independent evidence and useful decisions.**

We compare changes with training restrictions, latent information in a text VAE, and learned behavior. MG also selects forecast models from one neuron and RL policies under actuator shifts; strong scalar and held-out-error competitors remain in the comparisons.

4. **A fast sampled-query implementation.** On 20 new generator seeds, 128 queries preserve all 160 full-MG forecast choices with a 16.3-fold measured speedup. We separately report acquisition cost and comparisons with full-state diagnostics, including a 14–19-fold speedup over full Lyapunov spectra in the specified benchmark.

Delay embeddings and local dimension estimators are established tools. Our contribution is their systematic use for learning dynamics, with synthetic tests of the interpretation and independent application-specific evidence.

Relation to existing measurements. Intrinsic-dimension methods usually analyze clouds of points or representations (Facco et al., 2017; Ansuini et al., 2019; Pope et al., 2021); other work studies fractal dimensions of optimizer trajectories and invariant measures (Simsekli et al., 2020; Camuto et al., 2021; Birdal et al., 2021; Tan et al., 2024). PCA analyses of parameter trajectories provide a useful random-walk reference (Antognini and Sohl-Dickstein, 2018), while Koopman methods compare training dynamics through operator spectra (Redman et al., 2024). We instead estimate local temporal geometry from one scalar. Covariance participation ratio, Hessian spectra, and studies of representation, function complexity, and weight rank at grokking (Gao et al., 2017; Cohen et al., 2021; Liu et al., 2023; Yunis et al., 2024; Humayun et al., 2024) provide independent comparisons. Scalar roughness, autocorrelation, entropy, and surrogate records test what the geometric analysis adds.

2 DETECTING SIMPLIFICATION IN NEURAL NETWORK DYNAMICS

2.1 Problem statement

Given one scalar observation per step, we want to detect when the underlying dynamics become simpler. Let s_t be the system state and $x_t = \phi(s_t)$ the observation. Let P denote the number of trainable scalar parameters. During optimization, s_t includes these parameters and may also include optimizer variables; the ambient state dimension can therefore exceed P . When evaluating a trained recurrent generator, the weights are fixed and s_t is the evolving network activity. These are two different

trajectories to which we apply the same estimator.

We consider two tasks. In a recurrent regime with a well-defined dimension, we estimate the number of active components from a finite window of x_t . In nonstationary training, we test whether changes in the estimate track simplification measured independently of MG.

For the first task, let μ be an ergodic occupation measure and $B(s, \epsilon)$ a Euclidean ball in a fixed finite-dimensional state representation. Its measure describes the fraction of time spent near s . We define active dimension by

$$d_{\text{act}} = \lim_{\epsilon \downarrow 0} \frac{\log \mu(B(s, \epsilon))}{\log \epsilon}, \quad (1)$$

when the limit exists and has the same value μ -almost everywhere. Fixed invertible linear rescaling of state coordinates preserves this limit, although it can change finite-scale distances. On a smooth manifold with positive regular density, d_{act} equals its dimension. In particular, r independent oscillator phases need r coordinates even when the output has many harmonics.

For the second task, we hold the observable and estimator settings fixed across windows. Each experiment defines its reference before examining MG: restricted parameter updates, reduced latent information, or more regular learned behavior. These are measurable forms of simplification, but they need not share a single dimension or improve predictive performance. For example, losing useful latent information is simplification that a monitoring method should detect even though it is undesirable.

2.2 Delay reconstruction and dynamical-systems theory

A single observation can mix several components. Delayed observations can separate them by recording how the signal changes over time. We form the vector

$$y_t = (x_t, x_{t+\tau}, \dots, x_{t+(E-1)\tau}) \quad (2)$$

with E coordinates separated by lag τ . The Takens and Sauer results give conditions under which generic smooth observations distinguish the states of a deterministic system (Takens, 1981; Sauer et al., 1991). Stark’s extensions address forced systems under additional assumptions (Stark, 1999; Stark et al., 2003). These results explain why a scalar record can retain information about the underlying dynamics. They do not prescribe a finite-sample window or guarantee reconstruction for every SGD trajectory.

The condition needed for our dimension interpretation is precise. If the delay map is bi-Lipschitz on the sampled support, it changes distances by at most fixed

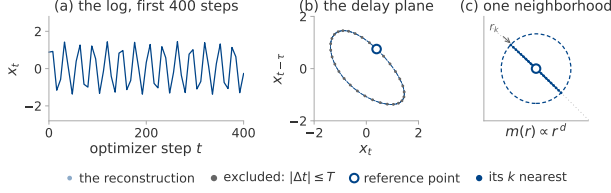


Figure 1: From a scalar record to the MG estimate. (a) The first 400 observations of a one-phase signal. (b) Two delayed coordinates reconstruct its closed-loop trajectory. (c) The estimator compares neighbor radii around a reference point after temporal exclusion; these radius ratios enter Eq. (3). The schematic explains the measurement geometry and is not an additional experiment.

factors. Those factors disappear in the limit in Eq. (1), so the local dimension is preserved. the supplement gives the statement and proof. We therefore estimate dimension from the delay vectors in recurrent systems with known structure, and test changes in the estimate against independent measures of simplification in applied training.

2.3 Pooled local-dimension estimation

Within a window of W scalar samples, we standardize the observations and form $n = W - (E - 1)\tau$ delay vectors. For each vector, we retain k nearest neighbors with time separation greater than the Theiler exclusion T (Theiler, 1986). Let r_{ij} be its Euclidean distance to the j th closest admissible neighbor, ordered increasingly. If every query has enough neighbors, we compute

$$\hat{d}_{\text{MG}} = \frac{n(k-1) - 1}{\sum_{i=1}^n \sum_{j=1}^{k-1} \log(r_{ik}/r_{ij})}. \quad (3)$$

The distance ratios measure how quickly neighborhoods fill as their radius grows; faster growth corresponds to higher local dimension. Equation (3) pools this information across query vectors. The subtraction of one is a finite-sample correction under the local likelihood model, which assumes approximately uniform density at the neighborhood scale. Serial dependence, uneven excitation, and short records can still introduce bias. We retain these effects in the reported estimates rather than clipping values to E .

Invariance to affine rescaling. For fixed delay and neighbor settings, the ideal estimator is invariant to $x_t \mapsto ax_t + b$, $a \neq 0$. Standardization removes the offset and amplitude scale, and the distance ratios remain unchanged. Thus MG can distinguish a change in tem-

poral structure from a uniform change in amplitude. the supplement proves this property and describes the effects of the small numerical noise added in the implementation. The invariance applies to a whole window; a scale change within that window can alter its temporal structure.

Window length and temporal resolution. The delay span $S = (E - 1)\tau$ determines the time interval represented by each vector. The window length W determines how much data enter an estimate and how precisely a transition can be located. Estimating dimension requires repeated visits within one regime. Monitoring requires a window short enough to distinguish the regimes before and after a change. We choose settings on development records and hold them fixed during evaluation, and report detection delay. the supplement lists the configurations.

2.4 Measurement and interpretation pipeline

The same computation serves two purposes. In a recurrent system with repeated returns, it can support an active-dimension interpretation when the embedding assumptions hold. In a general training log, it is a relative change score tied to the chosen observable and independent reference. The steps below make that distinction explicit:

1. **Define the measurement.** Specify the change of interest, an independent reference, and one scalar recorded at fixed intervals. Prefer an existing log; measure any extra probe cost. Choose the observable on development data, before examining test outcomes.
2. **Fix the measurement procedure.** Select W , stride, E , τ , k , T , and any detection threshold using development records. Fix either their values or a stated adaptive rule. Use identical rules for matched conditions and evaluate on held-out runs.
3. **Estimate and check.** Apply Eq. (3) to completed windows, time-stamped at their ends. Record raw amplitude and numerical-floor flags. Do not interpret flat or noise-floor-dominated windows as dimensions. Check sensitivity to W , τ , and $2E$; report failures and missing estimates.
4. **Assign an interpretation.** Dimension estimation requires recurrence, a stable occupation measure, a dimension-preserving observation, and sufficiently many returns with approximately uniform local density above the noise scale. Otherwise, interpret a drop supported by the independent reference only as a change associated with the specified reference, reporting detection delay, false positives, and scalar baselines.

Ergodicity of learned generators is assumed, not proved by finite rollouts. An insufficient E prevents faithful dimension recovery; the finite-sample estimate can nevertheless exceed E . Neither stable estimates nor agreement with a reference proves all embedding assumptions. In SGD, changes in noise and temporal correlation can also affect MG.

3 SYNTHETIC EXPERIMENTS: RECOVERING COMPONENT STRUCTURE

We first ask whether MG recovers the component structure predicted by the theory. Analytic signals provide known dimensions; learning problems with known references test the effect of parameter updates; trained generators test whether a single neuron reveals the phase structure of a learned task. Section 4 then evaluates monitoring in applied learning problems.

3.1 Analytic oscillators and learning

Oscillatory signals with known dimension. In the analytic baseline, r diagonal entries of a matrix oscillate at rationally independent frequencies. The trajectory therefore has r independent phases. MG computed from a scalar observation tracks this dimension over $r = 1$ –8, with mean absolute error 0.27. This experiment tests dimension recovery on prescribed oscillations; the following experiments include parameter training.

Classifier training on fixed MLP features. We train a multilayer perceptron (MLP) with tanh activations on handwritten digits (Pedregosa et al., 2011), freeze its features, and train a linear classifier in a fixed parameter subspace. Quasiperiodic changes in the loss weights of data groups provide a known source of recurrent motion. We select estimator settings on separate seeds and ranks, then evaluate ranks 1, 3, 5, 8 on new seeds.

Every usable observable orders the held-out ranks correctly with these fixed settings. Parameter norm and fixed-probe loss stop varying when the learning rate is zero, confirming that their variation requires learning. The mean absolute error against the independently measured covariance reference is 0.90 across rank–seed–observable combinations. This reference checks the excitation of parameter directions; it is not an exact active-dimension label. The result supports accurate ordering in a fixed learning task without retuning the estimator for each rank. the supplement gives errors for individual observables and distinguishes this task from earlier driven-target tests.

Endogenous oscillations in a regression problem. We next test whether MG detects fewer modes when all parameters remain trainable. We train all 32 weights of a dense linear regressor using full-batch heavy-ball updates. Repeated Hessian eigenvalues form r groups; momentum one produces r independent oscillatory modes with a fixed target. Selective damping reduces four modes to two without changing the data or number of trainable parameters. Across three coordinate/probe seeds, loss-based MG falls from 4.97 to 2.56; these seeds share the same loss dynamics (the supplement). Multiplying the original loss by 0.01 leaves MG unchanged. The estimate therefore responds to the removal of modes in this experiment, while remaining insensitive to uniform rescaling. This is an endogenous oscillatory optimizer: without damping, it oscillates around the solution rather than converging.

3.2 Learned generators distinguish phases from harmonics

The next test asks whether one neuron’s activity reveals the component structure learned by a recurrent network. A 1,000-unit rate network is trained by FORCE with recursive least squares (Sussillo and Abbott, 2009) to generate one to four sinusoidal outputs. Feeding the outputs back into the network makes the learned read-out affect its dynamics. At evaluation, the weights are fixed and the network runs without the target signal.

We train seven tasks on each of five seeds: one to four independent frequencies (T1–T4), two or four harmonics of one frequency (H2, H4), and two independent frequencies with their harmonics (M4). Their target phase dimensions are 1, 2, 3, 4, 1, 1, 2, respectively. All 35 networks meet the output-fidelity criterion. A Lyapunov analysis checks the learned dynamics independently. We choose MG settings on the target signals before analyzing the networks and use neuron zero as the primary observable.

The harmonic references demonstrate the benefit of geometric analysis (Figure 2, Table 1). For every seed, MG is lower for H2 than T2 and increases from H4 to M4 to T4. The latter tasks have identical numbers of spectral peaks and similar covariance PR. MG therefore distinguishes their phase structure from one neuron’s record. It also separates T1 and T2 from the higher-dimensional tasks, but overestimates dimension for T3 and T4 and does not distinguish them at this record length.

3.3 FORCE training from chaos to periodic output

In a separate FORCE experiment, a 256-unit network learns to produce a figure-eight trajectory from ini-

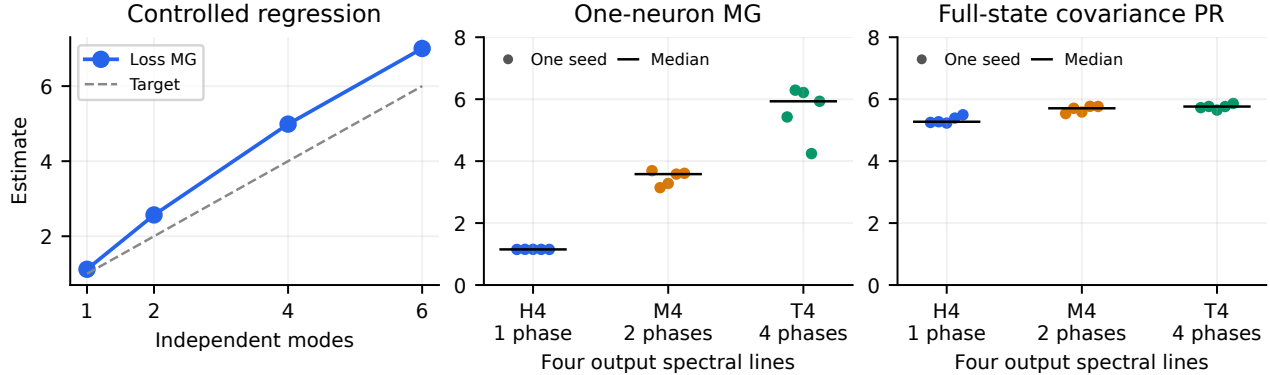


Figure 2: Phase structure recovered from scalar records. Left: regression with known modes; the dashed line is the target dimension. Center and right: generators with four output spectral lines and one (H4, blue), two (M4, orange), or four (T4, green) target phases. Dots represent seeds and black bars indicate medians. Single-neuron MG separates these phase structures despite similar full-state covariance participation ratios (PR).

Table 1: Learned generator results: medians over five seeds. Phases denotes target dimension; MG uses neuron zero and PR uses the full state. Each seed’s MG is the median over four 8,192-sample windows.

Task	Phases	MG	Covariance PR
T1	1	1.15	1.64
T2	2	2.84	3.01
T3	3	5.90	4.32
T4	4	5.93	5.76
H2	1	1.12	2.97
H4	1	1.15	5.27
M4	2	3.58	5.71

tially chaotic activity. The target has two outputs but one fundamental phase. We select five chaotic initializations using a pretraining Lyapunov criterion, without inspecting MG, and evaluate the network at saved checkpoints.

Single-neuron MG falls from 3.56–5.07 before training to 1.08–1.13 afterward in all five networks. The decline persists across three neurons and the tested window and lag settings. Independently, the full Lyapunov spectrum loses its clearly positive exponents. Four networks meet the stricter stable-cycle criterion; the fifth has weak contraction transverse to the cycle.

Both analyses identify simplification of the learned activity. MG does so from one neuron, while the spectrum uses the full state and tangent dynamics. Section 5 compares their computational costs. Here the measured trajectory is network activity at fixed weights, not the sequence of weight updates.

4 APPLICATIONS: MONITORING AND DECISIONS

We test scalar geometry against independent measurements of restricted training, latent information loss, and learned behavior, then use it to select policies and forecasters. Predictive utility is evaluated separately from absolute dimension interpretation.

4.1 CNN training: detecting restrictions on parameter updates

We train a 14,666-parameter convolutional neural network (CNN) on 10,000 CIFAR-10 images (Krizhevsky, 2009) for 10,000 SGD updates. At update 4,000, we reduce the learning rate, freeze layers, or prune weights. Reference runs continue unchanged or use a larger batch; separate transformed logs rescale or smooth the observation without altering training. MG receives only the norm of all parameters, which requires no extra network forward pass. Settings are $W = 1,000$, stride 500, $E = 20$, $\tau = 1$, and $k = 20$.

We choose detector settings on seeds 0–3 and test interventions of different strengths on new seeds 10–13. Independent measurements record the covariance participation ratio of updates, the fraction of parameters that move, and update magnitude. Freezing and pruning impose known restrictions on trainable or nonzero parameters. Learning-rate reduction changes the dynamics, but need not reduce invariant dimension; we assess its effect through the measured updates.

MG falls after strong restrictions on training. Freezing all but the classification head reduces it by a median 18.5%, while update participation ratio falls by 43.5%. Strong freezing and pruning produce median MG declines of approximately 12–19%; reference-run

medians range from near zero to increases of 13%. Across intervention strengths, MG ratios and update participation-ratio changes have Spearman correlation 0.59. They measure different aspects of the change: after 50% pruning, MG falls while update participation ratio rises.

MG separates interventions from unchanged-training and increased-batch reference runs with AUC 0.93, versus 0.48–0.57 for trend crossings, lag-one correlation, and detrended standard deviation. IAAFT spectral surrogates yield 0.68. the supplement specifies the descriptive surrogate check, all scalar competitors, and alternative comparison sets.

Causal detection of training interventions. For window estimates m_j , we compare the median over the latest M windows with the median over the preceding B windows:

$$D_j = \frac{\text{median}(m_{j-M+1}, \dots, m_j)}{\text{median}(m_{j-M-B+1}, \dots, m_{j-M})} - 1. \quad (4)$$

We choose $M = 2$, $B = 3$, and a detection threshold of $D_j < -0.0883$, a drop of 8.83%. Only completed windows enter the rule. The first threshold crossing defines a detected change. It is correct if it occurs within 5,000 updates after the intervention; an earlier crossing is a false positive. Competing statistics use the same selection procedure.

MG detects 27/28 strong interventions with 0/16 reference false positives, while normalized increments give 28/28 and 0/16 (Table 2). MG detects 8/28, 16/28, and 22/28 within 500, 1,000, and 2,000 updates. On ten additional seeds with shifted intervention times, it gives 19/30 detections and false positives in five of ten reference runs, versus 30/30 and zero for absolute increments. Thus detector transfer is task-specific; these paired records share seeds (the supplement).

4.2 Transferring the detector to ResNet-18

We transfer the parameter-norm observable, MG settings, and detection threshold to ResNet-18 (He et al., 2016), without redevelopment. Training uses the full CIFAR-10 training set, either from scratch or from ImageNet-pretrained weights, with three new seeds in each setting. Freezing the backbone while continuing to train the head reduces MG by a median 27.6% and 30.8%, respectively. The detector identifies all six freezing events with no false positives in 24 reference records.

This transfers a detector from about 15 thousand to 11 million parameters for a concrete training change. The global parameter norm is informative for backbone freezing, but the same detector misses the tested

Table 2: All scalar features with the same fixed block detector. Hits use a 5,000-update horizon. False positives are reference records with any threshold crossing; delay is the median over detected events. Four held-out seeds are shared across interventions and reference runs.

Feature	Detected	FP	Delay
MG	27/28	0/16	1,000
Abs. increments	27/28	0/16	500
Norm. increments	28/28	0/16	1,500
Perm. entropy	28/28	7/16	1,500
Sample entropy	28/28	4/16	1,500
Spectral entropy	4/28	2/16	2,750
Trend crossings	15/28	5/16	1,000
Lag-one corr.	0/28	0/16	—
Detrended std.	28/28	10/16	500
Level	4/28	0/16	1,000

learning-rate reductions and pruning interventions in ResNet-18. Separately explored layer norms and random projections introduce false positives. Thus transfer succeeds for the freezing task, while other interventions require further evaluation of the observable and detector.

4.3 Text VAE: tracking information loss and preservation

A text variational autoencoder (VAE) learns to reconstruct Penn Treebank sentences (Marcus et al., 1993) through a 16-dimensional Gaussian latent code. Its encoder and decoder use gated recurrent units (Bowman et al., 2016); the model has 276,016 parameters and trains on 6,000 sentences. This task tests whether MG detects a reduction in the dependence of the decoder’s output distribution on the latent code.

After 1,024 shared updates with Kullback–Leibler (KL) regularization weight $\beta = 0.01$, we split training into three branches: keep that weight, raise it to one, or use $\beta = 1$ with a free-bits objective. The free-bits objective excludes low-KL latent coordinates from further KL penalization. All branches train until update 3,072.

MG uses the reconstruction negative log-likelihood (NLL) per token on 16 fixed evaluation sentences, with fixed latent sampling noise. It has no access to KL or the independent references. On a larger set of 256 fixed evaluation sentences, the references estimate latent mutual information and the decoder’s response to shuffling codes between sentences. A joint decline means that less information is retained and used through the latent code.

The information-loss criterion is met on all nine test seeds. MG declines more strongly in the high-regularization branch than in the matched reference

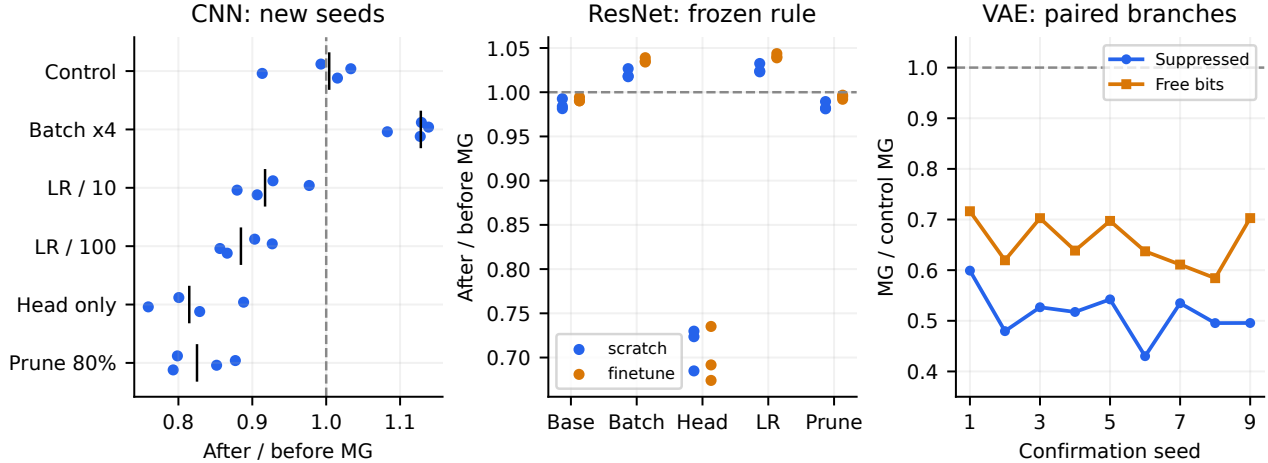


Figure 3: MG under training interventions and latent-information suppression. Left: post/pre-intervention ratios on four CNN seeds; vertical marks indicate medians. Center: corresponding ResNet-18 ratios for scratch and pretrained initialization. Right: late VAE estimates relative to the shared low-regularization reference run, for suppression and free-bits branches on nine seeds. Points represent seeds; dashed lines indicate a ratio of one. Independent update and information measurements provide the references.

on every seed: their median late MG ratio is 0.517, with a 95% seed-bootstrap interval [0.480, 0.542]. Free bits partially preserves both information and decoder dependence. MG is then higher than in the suppressed branch on all nine seeds, with median ratio 1.286 [1.179, 1.418]. The preservation reference is informative because the two branches use the same external KL weight but retain different amounts of information. MG distinguishes these outcomes from reconstruction logs alone.

This result validates MG as a signal of latent information loss and partial preservation. Standard deviation and spectral entropy also detect the loss on all nine seeds. In a separate cyclic-VAE test, using MG to schedule expensive reference checks finds 28/41 transitions at the primary budget, compared with 38/41 for periodic checks. Association with information loss therefore does not by itself imply better scheduling.

4.4 Grokking: a change in training dynamics

Here MG is used as a relative-change signal: the question is whether it detects a change in training dynamics, rather than whether it returns an exact dimension.

In modular-arithmetic training, scalar parameter-norm MG falls near generalization in all four positive runs when its temporal resolution is matched to that of the trajectory analysis. Independently, saved trajectory sketches show a sharp, reversible collapse of detrended covariance participation ratio. Across settings applied to these same runs, 24/26 usable comparisons distinguish the positive runs from two reference runs. These comparisons are not independent repetitions. Rough-

ness changes are stronger, a memorizing reference also simplifies, and the full-batch setting does not reproduce the same scalar result. The evidence supports sensitivity to a training transition, without making an MG decline a unique indicator of generalization.

4.5 Walker2d: selecting policies under actuator shifts

PPO policies in Walker2d (Schulman et al., 2017; Todorov et al., 2012) show lower action-log MG under moderate action smoothing in two five-seed series. This does not imply simpler whole-body motion. We additionally select among 16 saved policies per seed using three nominal rollouts and a shared reward/survival gate. On four held-out fine-tuning seeds, minimum action-norm MG gives deployment return 4.313 versus 4.005 for nominal-reward selection under hidden actuator lag/noise; completion rises from 80.0% to 90.8%. Spectral entropy performs better (4.480, 97.5%), and a pilot-selected fixed configuration nearly matches MG (4.310). This is exploratory policy selection, not demonstrated curriculum or training acceleration; the supplement reports all comparisons and shared-initial-policy limitations.

4.6 Selecting forecasters from one neuron’s activity

We test whether scalar geometry supports a useful decision: selecting a forecasting model for a trained recurrent network. A learned selector receives MG from 2,048 observations of one neuron, chooses among

Table 3: Forecast selection on 20 new seeds, eight tasks per seed. NMSE uses prefix variance. Time includes selection and the chosen fit on 12 matched inputs; a dash means not benchmarked. Acquisition and offline selector training are excluded.

Selector	NMSE	Time (ms)
MG	0.0364	199.1
MG, $q = 128$	0.0364	12.2
Held-out model search	0.0407	45.3
Cheap + held-out search	0.0338	—
Held-out errors + MG	0.0317	—

eight forecasters, and predicts the next 32 observations. The pool contains persistence, mean and cycle forecasts, linear regression, nearest-neighbor prediction, and a nonlinear feature model. We train the selector on 18 initialization seeds and freeze it before evaluation; network-training failures remain included; implementation details are in the supplement.

On 30 new seeds, normalized mean squared error (NMSE) is 0.0314 for MG selection, versus 0.0322 for validation search, 0.0354 for TwoNN, and 0.0405 for Levina–Bickel selection. The difference from validation is not statistically resolved. To reduce cost, we approximate MG using 128 query vectors while retaining the full neighbor cloud. On 20 further seeds, the approximation preserves all 160 full-MG model choices. Selection and fitting are 16.3 times faster than full MG and 3.7 times faster than validation search in the paired benchmark (Table 3). Stronger learned selectors using validation errors can be more accurate. In this setting, the sampled-query approximation makes model selection cheaper while preserving the full-MG choices. It does not show that MG improves every forecasting task or reduces the cost of training the underlying network.

5 COST, HARDWARE, AND CONDITIONS OF USE

Storage and analysis of scalar records. The main computational advantage is the input: MG processes one scalar per step. Once that log is available, it needs no weights, activations, or Jacobian products. Its storage and nearest-neighbor calculations depend on W , E , and the temporal exclusion, not on the number of network parameters. Obtaining the log has a separate cost: a parameter norm requires an $O(P)$ reduction over P parameters, and a fixed-probe loss requires a forward pass. We therefore report acquisition and analysis separately.

For the CNN, the scalar record occupies about 39 KiB, compared with 559 MiB for the stored float32 parameter trajectory. Table 4 reports analysis times. Stream-

Table 4: Analysis time, excluding scalar acquisition. References: full Lyapunov spectrum (FORCE; pre/post-training medians) and information/code-shuffling measurements (VAE). Comparisons are within rows; the supplement documents hardware and timing procedures.

Setting	MG	Reference
CNN, 1,000-point window	9.5 ms	—
FORCE, 256 units	0.60–0.84 s	11.55–12.01 s
VAE, 512-point window	12.89 ms	209.32 ms

ing covariance or sketches would reduce the reference storage; the comparison uses the saved full trajectory.

When analysis saves computation. In the 256-unit FORCE benchmark, MG and the full Lyapunov spectrum identify the learned transition toward a cycle. The measured speedup is 14–19-fold, or 5–10-fold including an embedding-doubling check. A 128-oscillator test gives 22–23-fold savings. These comparisons depend on window length and implementation; elementary scalar features and the largest exponent can be faster. MG does not recover individual exponents (the supplement).

In forecast selection, sampled-query MG reduces computation while retaining full-MG decisions on the held-out cohort (Section 4.6). Conversely, VAE probe acquisition dominates analysis: cyclic monitoring costs an estimated 54.68 s versus 2.72 s for periodic checks; reusing the probe reduces MG’s total to 3.54 s. Low analysis cost is most useful when the informative scalar is already available.

Observable sensitivity and detector transfer. Observable sensitivity limits transfer: in the ReLU-death test, the CNN detector gives 12/21 detections and 12/31 false positives, versus 19/21 and 7/31 for trend crossings. MG also fails to track near-threshold Hessian modes in the tested edge-of-stability setting. Section 2.4 therefore requires a separate check for each observable and event type; noise, drift, and nearly constant records remain possible confounders.

Conclusion. We introduced a scalar-log procedure grounded in delay-embedding theory. Synthetic recurrent experiments recover the expected component ordering, while applied experiments use MG as a relative-change statistic alongside independent measures. With recurrence and adequate delay coverage, MG can reflect active components; in general training logs, its meaning remains tied to the chosen observable and reference. Once a scalar is available, MG supports inexpensive decisions: sampled queries preserved all 160 full-MG forecast choices with a 16.3-fold measured speedup.

AI USE STATEMENT

The authors formulated the research questions, hypotheses, and scientific claims. Generative AI tools assisted with conducting experiments, including implementation, debugging, analysis, and preparation of figures and reports. They also assisted with drafting, translation, and editing. The authors checked all reported results, read the complete manuscript, and further revised the text. The authors take responsibility for the final text, claims, code, and experimental results.

REFERENCES

- Alessio Ansuini, Alessandro Laio, Jakob H. Macke, and Davide Zoccolan. Intrinsic dimension of data representations in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Joseph M. Antognini and Jascha Sohl-Dickstein. PCA of high dimensional random walks with comparison to neural network training. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- Tolga Birdal, Aaron Lou, Leonidas Guibas, and Umut Simsekli. Intrinsic dimension, persistent homology and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.
- Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 10–21, 2016. doi: 10.18653/v1/K16-1002.
- Alexander Camuto, George Deligiannidis, Murat A. Erdogdu, Mert Gurbuzbalaban, Umut Simsekli, and Lingjiong Zhu. Fractal structure and generalization properties of stochastic optimization algorithms. *arXiv preprint arXiv:2106.04881*, 2021.
- Jeremy M Cohen, Simran Kaur, Yuanzhi Li, J Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=jh-rTtvkGeM>.
- Elena Facco, Maria d’Errico, Alex Rodriguez, and Alessandro Laio. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific Reports*, 7(1):12140, 2017.
- Peiran Gao, Eric Trautmann, Byron M. Yu, Gopal Sathnam, Stephen Ryu, Krishna Shenoy, and Surya Ganguli. A theory of multineuronal dimensionality, dynamics and measurement. *bioRxiv* 214262, 2017. doi: 10.1101/214262.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- Ahmed Imtiaz Humayun, Randall Balestrieri, and Richard Baraniuk. Deep networks always grok and here is why. *arXiv preprint arXiv:2402.15555*, 2024.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Elizaveta Levina and Peter J. Bickel. Maximum likelihood estimation of intrinsic dimension. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 17, 2004. URL <https://proceedings.neurips.cc/paper/2004/file/74934548253bcab8490ebd74afecebb5-Paper.pdf>.
- Ziming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=zDiHoIWa0q1>.
- David J. C. MacKay and Zoubin Ghahramani. Comments on ‘maximum likelihood estimation of intrinsic dimension’ by E. Levina and P. Bickel, 2005. URL <https://www.inference.org.uk/mackay/dimension/>. Unpublished note.
- Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.
- Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Phil Pope, Chen Zhu, Ahmed Abdelkader, Micah Goldblum, and Tom Goldstein. The intrinsic dimension of images and its impact on learning. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=XJk19XzGq2J>.
- William T. Redman, Juan Bello-Rivas, Maria Fonoberova, Ryan Mohr, Yannis G. Kevrekidis, and Igor Mezic. Identifying equivalent training dynamics. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. doi: 10.52202/079017-0744.
- Tim Sauer, James A. Yorke, and Martin Casdagli. Embedology. *Journal of Statistical Physics*, 65(3-4): 579–616, 1991.

- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.
- Umut Simsekli, Ozan Sener, George Deligiannidis, and Murat A. Erdogdu. Hausdorff dimension, heavy tails, and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5138–5151, 2020.
- Jaroslav Stark. Delay embeddings for forced systems. I. Deterministic forcing. *Journal of Nonlinear Science*, 9(3):255–332, 1999. doi: 10.1007/s003329900072.
- Jaroslav Stark, David S Broomhead, ME Davies, and J Huke. Delay embeddings for forced systems. II. Stochastic forcing. *Journal of Nonlinear Science*, 13(6):519–577, 2003. doi: 10.1007/s00332-003-0534-4.
- David Sussillo and L. F. Abbott. Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4):544–557, 2009. doi: 10.1016/j.neuron.2009.07.018.
- Floris Takens. Detecting strange attractors in turbulence. In *Dynamical systems and turbulence, Warwick 1980*, pages 366–381. Springer, 1981. doi: 10.1007/BFb0091924.
- Charlie B. Tan, Ines Garcia-Redondo, Qiquan Wang, Michael M. Bronstein, and Anthea Monod. On the limitations of fractal dimension as a measure of generalization. *arXiv preprint arXiv:2406.02234*, 2024.
- James Theiler. Spurious dimension from correlation algorithms applied to limited time-series data. *Physical Review A*, 34(3):2427–2432, 1986.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.
- David Yunis, Kumar Kshitij Patel, Samuel Wheeler, Pedro Henrique Pamplona Savarese, Gal Vardi, Karen Livescu, Michael Maire, and Matthew Walter. Grokking, rank minimization and generalization in deep learning. In *ICML 2024 Workshops: MI*, 2024.

CHECKLIST

1. For all models and algorithms presented, check if you include:
 - (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes] Sections 2–4 and the appendices.
 - (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes] Scale invariance, window requirements, storage and empirical cost are discussed; no universal sample-complexity bound is claimed.
 - (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [No] The bundle includes analysis scripts and numerical evidence, but not the complete training software with all dependencies.
2. For any theoretical claim, check if you include:
 - (a) Statements of the full set of assumptions of all theoretical results. [Yes] The propositions state their assumptions. Embedding results are cited as motivation and are not claimed to hold for every training record.
 - (b) Complete proofs of all theoretical results. [Yes] the supplement proves the two elementary propositions stated here; established embedding theorems are cited.
 - (c) Clear explanations of any assumptions. [Yes] Section 2 and the supplementary material distinguish geometric interpretation from empirical monitoring.
3. For all figures and tables that present empirical results, check if you include:
 - (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [No] Saved outputs and scripts reproduce tables and figures; the materials required to repeat every training run are not fully bundled.
 - (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Yes] Experiment-specific descriptions and development splits are in the appendices and accompanying procedures.
 - (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes] Captions and the supplement specify statistics, replication units, and the VAE bootstrap.
 - (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [No] the supplement documents recovered CPU, software, and thread settings; hardware metadata are incomplete for some historical runs.
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
 - (a) Citations of the creator If your work uses existing assets. [Yes] Dataset/model and methodological sources are cited in the text and asset notes.
 - (b) The license information of the assets, if applicable. [No] A consolidated license inventory is not included in this version.
 - (c) New assets either in the supplemental material or as a URL, if applicable. [Yes] Numerical evidence and manuscript-reconstruction scripts are provided with the source bundle.
 - (d) Information about consent from data providers/curators. [Not Applicable] No newly collected participant data are used.
 - (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Not Applicable] No new textual or image dataset is released; the bundled evidence is numerical.
5. If you used crowdsourcing or conducted research with human subjects, check if you include:
 - (a) The full text of instructions given to participants and screenshots. [Not Applicable]
 - (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable]
 - (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable]